Heart disease prediction: data to BITS deployment

Student ID: 2025AE05568
Prepared: 18 October 2026 (IST)

An educational classifier based on original UCI Cleveland data. The workflow combines deterministic data validation, training-only cross-validation, MLflow tracking, a packaged preprocessing/model pipeline, GitHub automation, Docker serving and monitored Kubernetes deployment on the BITS VM.

Measure	Selected experiment
Data split	242 training / 61 held-out rows
Selected family	logistic_regression
Hyperparameters	{"C": 0.1, "class_weight": "balanced"}
Mean CV ROC-AUC	0.9110 +/- 0.0305
Held-out accuracy / ROC-AUC	0.8852 / 0.9654
BITS verification	2026-10-18T08:17:37.756316+00:00

Repository: https://github.com/2025ae05568-rkmishra/mlops-assignment1
Release commit: ae005d121706927a0d7956126c87bf5c85ada6c0

The original Cleveland dataset contains 13 predictors plus the diagnosis target. Diagnosis codes 1-4 become the positive class and code 0 the negative class. The model is educational; its scores are not clinically validated risk.

Architecture and environment

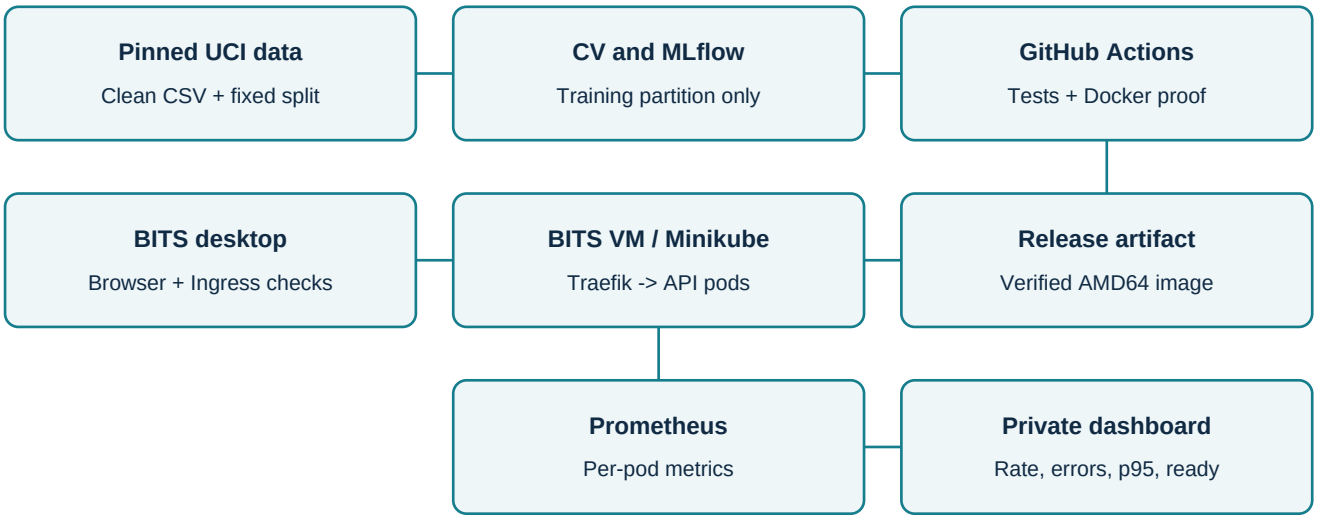


Figure 1. Training, release, private Kubernetes access and monitoring flow.

The build runs in GitHub Actions on AMD64. The BITS VM downloads the successful release artifact, checks the archive checksum, loads the image into Minikube and verifies predictions through Traefik Ingress. Training and image rebuilding are not needed on the VM.

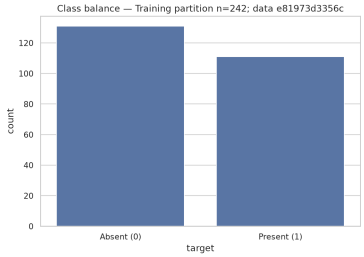
The demonstrated access path is the authorized BITS remote desktop. A persistent VM terminal forwards localhost port 8081 to Traefik; Host heart-disease.test selects the Ingress rule. The browser opens the private dashboard and MLflow on the VM. This is the permitted local-Kubernetes deployment option.

Boundary	Implementation
Training	Python 3.11; locked requirements; scikit-learn and MLflow
Serving	FastAPI/Uvicorn; one worker per pod; verified baked-in model
Deployment	Rocky Linux 9 x86_64; Minikube 1.35; Kubernetes 1.32.0
Capacity	Two API replicas on one node; no node-level fault tolerance
Monitoring	Per-pod Prometheus scraping; private four-panel dashboard

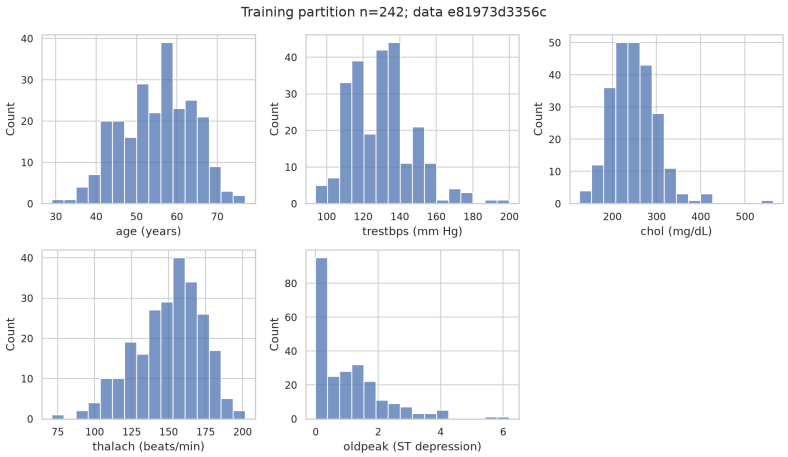
Data provenance, cleaning and exploratory analysis

Source: Janosi et al. (1989), Heart Disease, UCI Machine Learning Repository, DOI 10.24432/C52P4X, CC BY 4.0. Source archive and Cleveland member hashes are pinned. The binary target is int(num > 0); source category codes are retained.

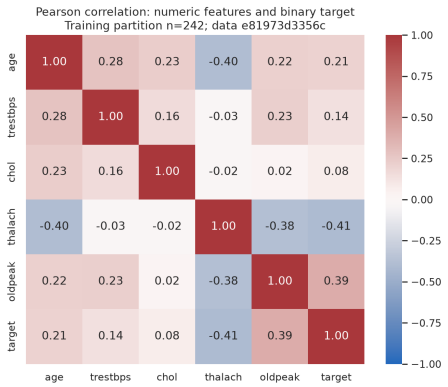
Observed cleaning: 303 source rows; 303 retained; 0 invalid; 0 duplicates removed; 0 conflicting rows. Missing values by feature: {"ca": 4, "thal": 2}. Imputation is fitted within training folds.



Training-partition class balance.



Training-only continuous-feature distributions.



Training-only correlation heatmap; associations are descriptive, not causal.

The training classes are only moderately imbalanced. Oldpeak is right-skewed, and maximum heart rate decreases with age in this cohort. Missing ca/thal values are retained for fold-fitted imputation; no held-out data are used to choose transformations.

Feature engineering, model comparison and tracking

A fixed stratified 80/20 split and five shared training-only folds use seed 42. Numeric features use median imputation and scaling for Logistic Regression. Categorical features use mode imputation and one-hot encoding. Random Forest omits scaling. The entire fitted pipeline is shared with inference.

Continuous inputs are age, trestbps, chol, thalach and oldpeak. The eight remaining source-code fields are categorical; their numeric codes are not treated as continuous magnitudes.

The search evaluates 25 configurations: one prior-class baseline, six Logistic Regression settings (C=0.1, 1, 10; class weight none/balanced), and 18 Random Forest settings (200 trees; depth unrestricted/4/8; minimum leaf 1/3/5; weight none/balanced). Selection uses mean CV ROC-AUC, preferring Logistic Regression within 0.01 of the best. Threshold 0.5 is fixed; held-out results do not drive tuning.

Family	CV accuracy / precision / recall / AUC
dummy	0.5413 / 0.0000 / 0.0000 / 0.5000
logistic_regression	0.8267 / 0.8379 / 0.7850 / 0.9110
random_forest	0.8103 / 0.8399 / 0.7403 / 0.9110

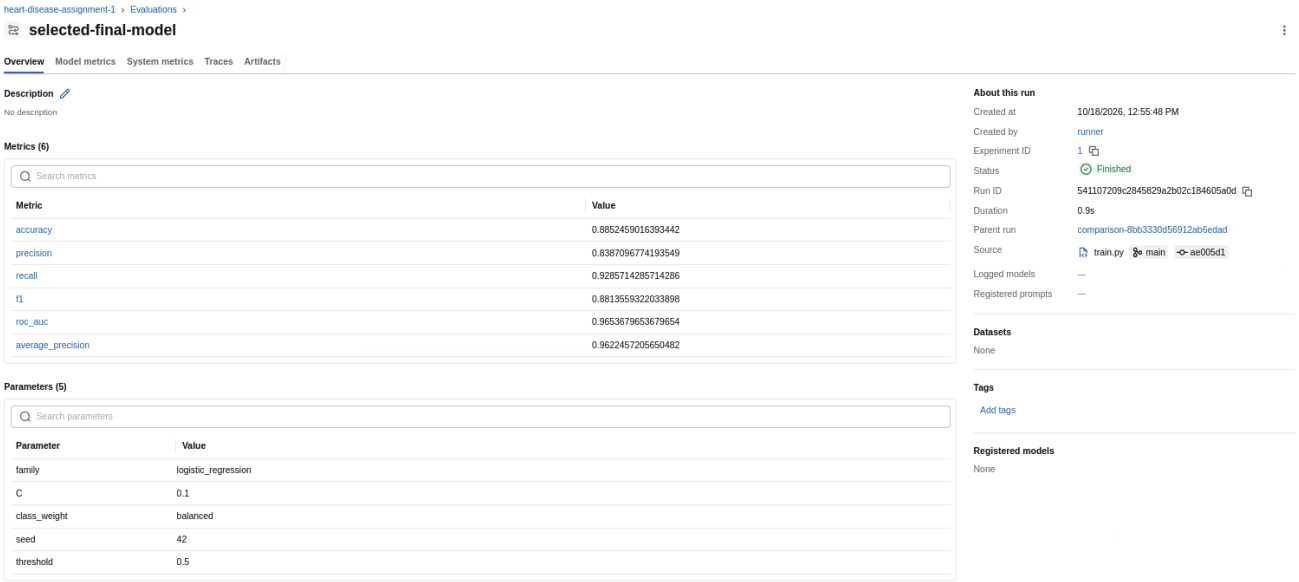


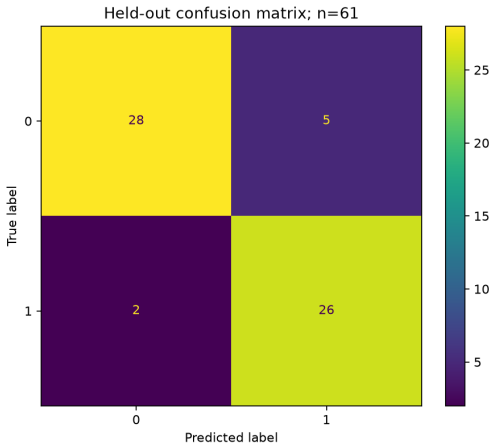
Figure 2. Selected MLflow run: held-out metrics, parameters and parent-run lineage.

Each candidate child run logs fold metrics, aggregate mean/std, parameters, schema, lineage and a CV plot. The final refit logs held-out curves and the complete serving bundle. The MLflow run ID is 541107209c2845829a2b02c184605a0d. Artifact screenshots and original plots are retained with the evidence.

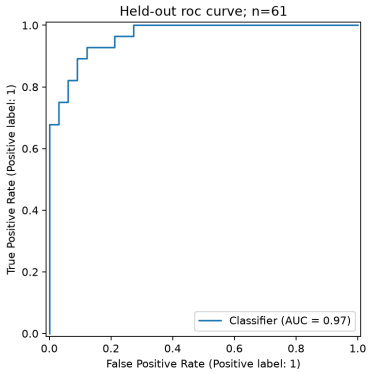
Held-out results and limitations

After selection, the complete pipeline is fitted on 242 training rows and evaluated on 61 held-out rows. Repeated fixed-split results are reproducibility evidence, not independent validation.

Held-out metric	Value
accuracy	0.885246
average_precision	0.962246
f1	0.881356
precision	0.838710
recall	0.928571
roc_auc	0.965368



Held-out confusion matrix: 28 true negatives, 26 true positives, five false positives and two false negatives.



Held-out ROC curve; ranking quality is distinct from calibrated risk.

The small historical cohort limits generalizability. There is no external-cohort validation or prospective clinical evaluation. Class-weighted probabilities are not calibrated clinical risk.

Packaging, API and isolated container verification

The serving bundle contains fitted preprocessing and classifier, feature schema, metrics, model card, manifest and checksums. Startup verifies integrity, classes and feature order before readiness. The image contains the bundle and does not depend on MLflow availability at inference time.

Endpoint / response	Contract
POST /predict	All 13 fields required; ca and thal allow explicit null
Prediction	Binary label, positive-class probability and returned-class confidence
Version	Model/schema version and request ID accompany the prediction
Validation	Unknown codes, extra/missing fields and invalid types return 422
Availability	/health liveness; /ready verified model; unavailable model returns 503
Boundaries	16 KiB request limit; sanitized JSON logs; /metrics internal only

Docker runs as UID 10001 with read-only root and writable /tmp. The release workflow builds and tests the AMD64 container with synthetic valid/invalid input, verifies the served model version, and saves isolated-serving evidence.

Model version: 8bb3330d56912ab6edad-54110720
Image: heart-disease-api:ae005d121706927a0d7956126c87bf5c85ada6c0-39107510102-1
Image identity: sha256:89c2819a7a544406b527fcb187d166821a63c06c9992d2fdac30738c3804422f

Captured Ingress result: prediction 0, positive-class probability 0.225527, confidence 0.774473; invalid status 422; external metrics status 404.

Synthetic JSON input:
{"age":52,"sex":1,"cp":2,"trestbps":128,"chol":210,"fbs":0,"restecg":0,"thalach":165,"exang":0,"oldpeak":0.8,"slope":2,"ca":null,"thal":3}

Confidence is p for a positive prediction and 1-p for a negative prediction; the returned label uses threshold 0.5. The API includes a request ID so a response can be traced to structured logs.

CI automation and release provenance

GitHub Actions runs lint, unit tests, data preparation, EDA, model comparison/training, packaging, integration tests and Docker smoke tests. Pipefail prevents failed checks from being masked by tee. Artifact upload runs with if: always() to preserve available failure diagnostics.

Workflow run: https://github.com/2025ae05568-rkmishra/mlops-assignment1/actions/runs/39107510102

Verified release test counts: 31 unit and 2 integration tests, zero failures/errors/skips.

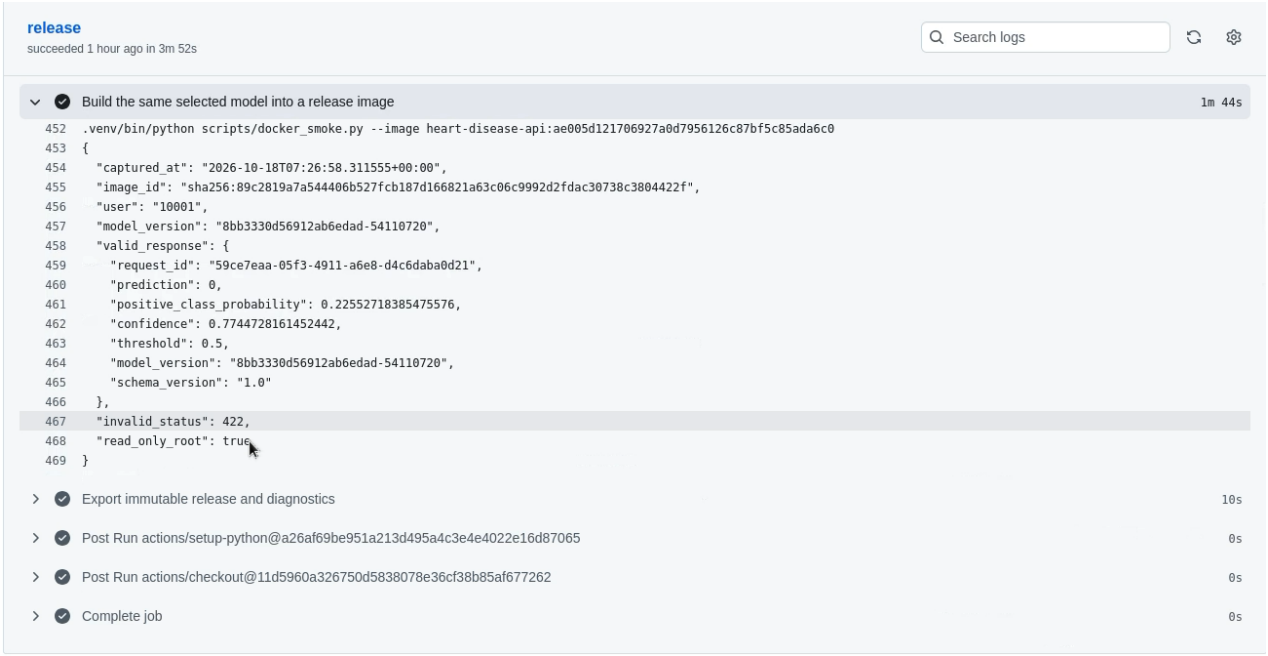


Figure 3. Successful release job: isolated Docker smoke test and matching model/image IDs (recording at 02:00).

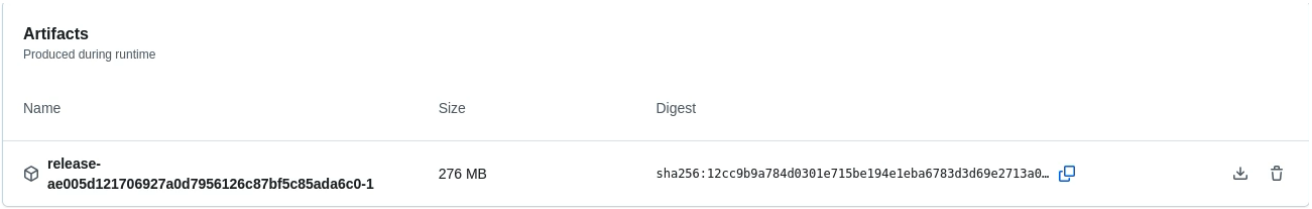


Figure 4. The same successful workflow exposes its downloadable release artifact (recording at 11:00).

The VM verifies the downloaded image archive checksum before loading. Release commit, run/attempt, model version, archive hash, image identity and actual pod resources are retained together. Responses identify the selected release. The model manifest preserves dirty=true and source/config/data hashes rather than asserting a clean tree after generated outputs.

Archive SHA-256: 65d2f2700da076df032795cffc347cdef2b7ab87f97eaa38f048768e2df07f45

BITS Kubernetes deployment and access

The application uses a Deployment with two replicas, startup/readiness/liveness probes, resource requests/limits and a ClusterIP Service. Traefik interprets Exact-path Ingress rules for prediction, health, readiness, version and API docs; /metrics is excluded.

The VM's Minikube 1.35 has no Traefik addon. Applying deploy/traefik/controller.yaml installed the pinned controller without upgrading host tools. The successful release was then redeployed with --reuse-run 39107510102. The final script retains this manifest-based installation path.



Figure 5. Successful BITS deployment, selected release identity and valid/invalid Ingress responses.

Local testing access (demonstrated on the BITS desktop):

The API is deployed privately to Minikube on the BITS Rocky Linux 9 VM; there is no public URL. In the BITS VM desktop terminal, enter the repository at ~/workspace/mlops-assignment1 and run: bash scripts/open_vm_access.sh. Keep that terminal open. The dashboard is at http://127.0.0.1:8090, the selected release's MLflow view is at http://127.0.0.1:5000, and the API Ingress is at http://127.0.0.1:8081 with Host heart-disease.test.

In a second VM terminal, run: .venv/bin/python scripts/smoke_api.py --url http://127.0.0.1:8081 --host heart-disease.test. The valid example returns prediction 0, model version 8bb3330d56912ab6edad-54110720 and confidence 0.774473; invalid input returns HTTP 422. Press Ctrl-C in the access terminal when finished. The Kubernetes deployment remains running.

Keep the access terminal running during testing. The forward targets the Traefik controller Service and exercises the host/path Ingress rules.

Monitoring and request logging

Prometheus discovers each API pod using namespace-scoped read-only RBAC and scrapes every five seconds. The private dashboard shows prediction request rate, 4xx/5xx rate, model inference p95 and ready model replicas. No online accuracy or drift measurement is claimed.

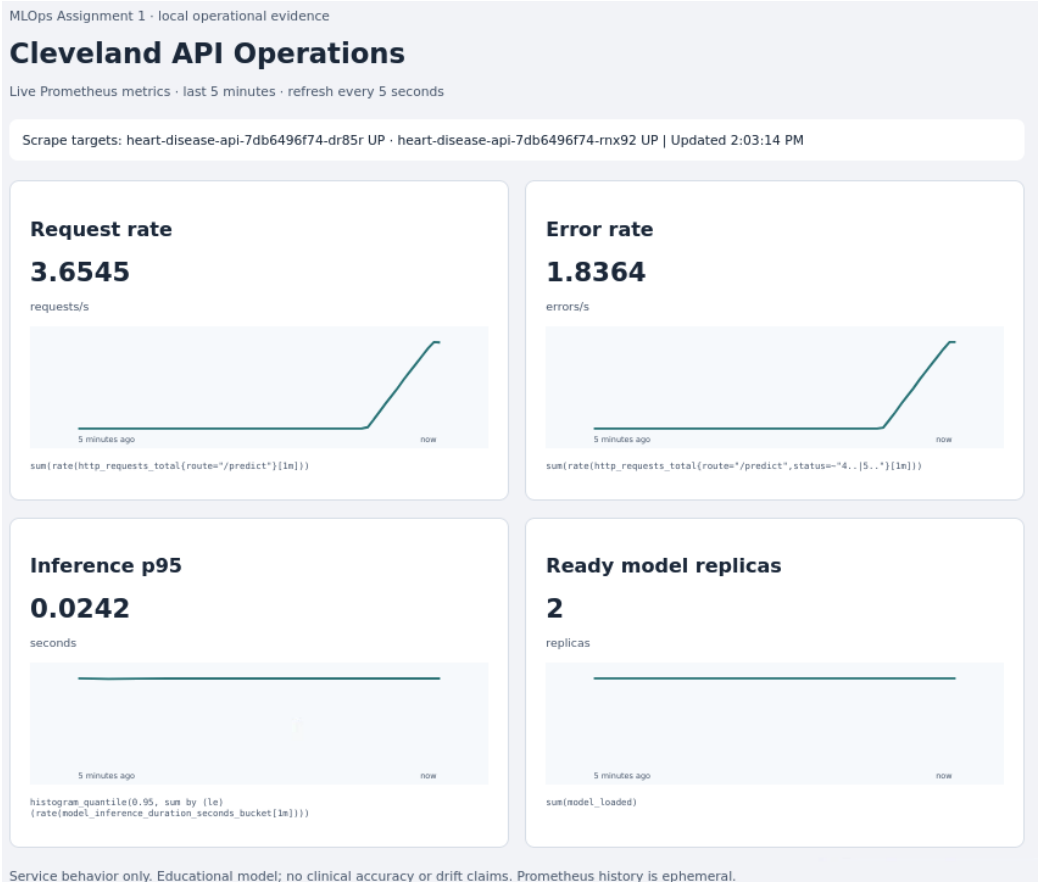


Figure 6. BITS dashboard after additional manual traffic; both current pod targets are UP.

Captured synthetic traffic: {"200": 57, "422": 57} over 30 seconds. Expected invalid requests deliberately contribute to the 4xx series.

The dashboard screenshot was taken later than this automated capture, after a further 60-second test (111 successful and 111 rejected requests). Its higher rate is expected; HTTP 422 is deliberate validation traffic, not a server failure.

Structured logs retain request ID, route, status, duration and model version without raw health values. Captured logs are included in api-logs.txt. Metric labels are bounded. Inference p95 measures model computation rather than network latency.

Prometheus retains up to 24 hours / 256 MB using ephemeral storage. Export the evidence directory before stopping or replacing the VM. Two API replicas on the same node do not protect against VM failure.

Setup, reproduction and submission deliverables

Clean development setup: clone the repository linked on page 1 and enter its directory. With Python 3.11 available, run **make setup** (locked requirements and editable package), then **make verify** (lint, data, EDA, training, packaging and tests). Run **make docker-build docker-smoke** to repeat isolated serving checks. Detailed commands: README.md.

BITS deployment: Docker and Minikube must be available to the regular user. Run **bash scripts/bootstrap_rocky9.sh --skip-installed**, authenticate with **gh auth login**, then run **bash scripts/run_lab_deployment.sh** from a clean checkout matching pushed main. The runner builds on GitHub and deploys the verified artifact. **--reuse-run** accepts a successful release of that same current main commit; after submitting later commits, use the default command for a new deployment. This report retains evidence for the recorded release on page 1.

Report and evidence: the completed ten-page PDF is provided in output/pdf. Its supporting metrics, plots, screenshots, deployment checks and local access instructions are preserved in reports/evidence/bits. Submit the existing MP4 separately with the coursework.

Assignment task locations (paths are relative to the repository root):

Task	Code and saved evidence in Git
1. Data acquisition and EDA	src/heart_disease/data/acquire.py, prepare.py, eda.py; data/README.md; data/processed/cleaned.csv; data/manifests/; reports/evidence/bits/figures/
2. Features and model selection	src/heart_disease/features/; src/heart_disease/models/; configs/training.yaml; reports/evidence/bits/metrics/
3. Experiment tracking	src/heart_disease/models/train.py; reports/evidence/bits/screenshots/mlflow*.png; GitHub Actions release artifact (MLflow export)
4. Packaging and reproducibility	src/heart_disease/models/package.py; requirements*.txt; Dockerfile; GitHub Actions release artifact (saved pipeline)
5. Tests and CI	tests/; .github/workflows/ci.yml; .github/workflows/release.yml; reports/evidence/bits/ci/
6. Docker API	Dockerfile; src/heart_disease/api/; scripts/docker_smoke.py; reports/evidence/bits/ci/docker-smoke.json
7. Kubernetes deployment	deploy/k8s/; deploy/traefik/controller.yaml; scripts/run_lab_deployment.sh; reports/evidence/bits/screenshots/deployment.png
8. Monitoring	deploy/monitoring/; scripts/monitoring_dashboard.py; reports/evidence/bits/api-logs.txt; reports/evidence/bits/screenshots/monitoring.png
9. Documentation	README.md; this report; reports/evidence/bits/reviewer-access.md; reports/evidence/bits/screenshots/

The saved model and full MLflow records are generated in the successful release workflow artifact (linked on page 7); they are not checked into the source tree.

Video: 2025ae05568-mlops.mp4 (submitted separately; 11 minutes 13 seconds).

Sources and assistance: Janosi et al., UCI Heart Disease, https://doi.org/10.24432/C52P4X (CC BY 4.0); repository and selected workflow linked above. Implementation and documentation were developed with Codex assistance. Individual understanding and compliance with course rules remain the student's responsibility.